

Clothing Classification on Fashion-MNIST

Decision Trees, Ensembles, and a Deep Learning Outlook

Thijs Niekerk (SNR: 2125613)

Pjotr Martens (SNR: 2080478)

Tilburg School of Economics and Management
Tilburg University

September 24, 2026

1 Data and setup

We use the Fashion-MNIST dataset (Xiao et al., 2017): 70,000 grayscale 28×28 images of ten clothing categories, split into 60,000 training and 10,000 test images with an exactly balanced class distribution (Figure 2). Each image is flattened into a 784-dimensional pixel-intensity vector, rescaled to $[0, 1]$, with no further preprocessing. Sample images are shown in Figure 1. All models train on the full training set and are evaluated once on the untouched test set; a stratified 10% validation split (6,000 images) is held out instead for hyperparameter selection. We report accuracy alongside macro-averaged F1, since the classes are balanced but not equally easy, so macro F1 does not let strong classes hide weak ones.

2 Part 1: decision tree and ensemble method

2.1 Decision tree (1a)

We fit a `scikit-learn DecisionTreeClassifier` (Pedregosa et al., 2011) using Gini impurity and greedy axis-aligned splits on individual pixels. The only hyperparameter tuned is `max_depth`. Figure 3 plots training and validation accuracy for depths 4, 6, $\dots$, 25 and an unconstrained tree: training accuracy reaches 100% once unpruned, while validation accuracy peaks at depth 12 (82.1%) and then falls off, the usual overfitting pattern. We fix `max_depth`= 12 and refit on all 60,000 training images, giving 1,200 leaves; Table 1 reports the result.

Model	Test accuracy	Macro F1	Fit time
Decision tree (depth 12)	80.3%	0.803	21.9 s

Table 1: Test-set performance of the single decision tree.

Test accuracy (80.3%) comes in a bit below the 82.1% seen during depth selection, even though the final tree saw more data. We read this as sampling variance rather than a mistake: a single tree is a high-variance model (Section 2.2), so two same-depth trees fit on different subsets can reasonably differ by a point or two. It is itself an early sign of the variance problem addressed in Part 1b.

Performance is far from uniform across classes (Figure 5): the tree separates *Trouser* (F1 0.95), *Bag* (0.92) and *Sandal* (0.91) almost perfectly, but is weak on *Shirt* (0.52), *Coat* (0.67) and *Pullover* (0.68). This is not random noise: errors concentrate almost entirely within {T-shirt/top, Pullover, Coat, Shirt}, four upper-body garments with a similar silhouette at 28×28 resolution. A decision tree only ever asks whether one pixel is above a threshold, so it can separate classes that differ in coarse shape or brightness (a trouser’s two legs, a bag’s handle) but has no way to represent the finer texture that separates a shirt from a pullover. This is a bias problem tied to the raw-pixel representation, not something more trees alone will fix.

2.2 Ensemble method (1b)

Choice and motivation. The depth curve in Figure 3 also shows a second problem: an unconstrained tree hits 100% training accuracy but only 80.1% on validation, a 19.9-point gap that signals high variance on top of the bias problem above. Ensembling is the standard fix for variance, and we use *bagging*, specifically a **random forest**, rather than a boosting method such as AdaBoost or gradient boosting. Bagging averages trees fit on bootstrap samples and decorrelates them further through random feature subsampling, which targets variance directly without changing what any single tree can represent. It also grows trees independently, so

training parallelises cheaply even with hundreds of trees on 784 features. Boosting instead reduces bias by fitting sequentially to the residual error, which makes it more sensitive to noisy, correlated features and to how many rounds are used, a real risk here given 784 correlated pixels and limited project time. To check this reasoning we also trained a histogram-based gradient boosting model (`HistGradientBoostingClassifier`, default settings) purely as a reference, not as our chosen ensemble.

Implementation. We tune the number of trees over $\{10, 25, 50, 100, 200\}$ on the same validation split (Figure 4); accuracy rises quickly and then flattens, so we pick 200. The final forest is refit on all training data with out-of-bag (OOB) scoring on, which estimates test performance from the roughly one third of images each tree never sees. The OOB accuracy, 88.4%, sits close to the actual test accuracy in Table 2 (87.8%), a reasonable internal check against overfitting.

Model	Test accuracy	Macro F1	Fit time
Decision tree (depth 12)	80.3%	0.803	21.9 s
Random forest (200 trees)	87.8%	0.877	90.0 s
Gradient boosting (reference)	89.5%	0.894	155.7 s

Table 2: Test-set performance of the tree-based models.

Expected versus observed effects. We expected the ensemble to cut variance and raise accuracy without touching the bias identified in Part 1a, and both hold. Accuracy rises from 80.3% to 87.8%, a 38% relative drop in test error, concentrated where the tree was weakest (Figure 6): F1 for *Shirt* goes from 0.52 to 0.65, *Coat* from 0.67 to 0.79, *Pullover* from 0.68 to 0.78, while an easy class like *Trouser* barely moves (0.95 to 0.98). The same four classes still dominate the confusion matrix, just less often: averaging pixel-threshold trees cannot invent spatial features none of them had access to. The pixel-importance map (Figure 7) backs this up; importance spreads diffusely over the torso outline rather than sitting on a handful of pixels, matching how these classes differ mainly in fine texture rather than shape. Gradient boosting adds another 1.7 points (89.5%) at about 1.7 times the training time, the usual higher-cost, higher-ceiling trade-off of boosting; given the project’s scope we keep the random forest. With 10,000 test images the standard error on each accuracy is at most 0.4 points, so both gains are well beyond sampling noise. The full per-class breakdown for all three models is in Table 3, Appendix A.

Difficulties. Two things slowed the implementation down. A joint grid search over `max_depth` and `n_estimators` with proper cross-validation was too slow at this scale for the time available, so we tuned them one at a time on a single validation split instead, cheaper but not guaranteed to find the jointly best combination. A standard `GradientBoostingClassifier` was also too slow to serve even as a reference, so we used the histogram-based `HistGradientBoostingClassifier`, which approximates the same idea on binned features at a fraction of the cost.

3 Part 2: towards a deep learning approach

As the assignment specifies, this part is exploratory and design-only: nothing here is implemented.

3.1 Shortcomings observed in Part 1b (2c)

Two related shortcomings limit the tree-based approach, and neither goes away with more trees. Decision trees split on one raw pixel at a time, so neighbouring pixels are treated as unrelated: the model cannot learn edges, folds or textures, or be invariant to the small shifts and shape

changes present in the images. Second, with 784 largely redundant, correlated pixel features, greedy single-feature splits summarise the image inefficiently; importance ends up smeared across many similar pixels (Figure 7) instead of a compact set of reusable shape features. Ensembling reduces variance and helps for exactly this reason, but every tree in the forest still only sees raw pixels, so the bias persists, which is what shows up as the residual Shirt/Pullover/Coat/T-shirt confusion.

3.2 Proposed design: a convolutional neural network (2d)

Both problems point to the same fix: a model that exploits spatial locality and shares parameters across the image instead of treating every pixel independently. That is what a **convolutional neural network (CNN)** is built for. We propose a compact architecture: two convolution-plus-max-pooling blocks (e.g. 32 then 64 filters of size 3×3 , ReLU, 2×2 pooling), a flatten step, one dense layer with dropout, and a 10-way softmax output. The filters are learned rather than hand-designed, so the network finds its own edge and texture detectors; pooling adds some translation invariance and cuts spatial resolution before the dense layer turns the local features into a class prediction. Dropout, and optionally batch normalisation, between layers controls overfitting, playing a similar role to ensembling in Part 1b but by regularising one expressive model instead of averaging many high-variance ones.

3.3 Implementation plan and expected results (2e)

We would implement the network in PyTorch or Keras, train with Adam and cross-entropy loss on mini-batches, use the same 90/10 split as Part 1 for early stopping and hyperparameter choices (learning rate, filter counts, dropout), and evaluate once on the test set with the same metrics as Table 2 for a fair comparison. Published benchmarks on this dataset report that even small two-layer CNNs reach roughly 88–92% test accuracy (Zalando Research, 2026), on par with or above our gradient boosting reference and clearly above our random forest. Most of that gain should land on the classes our tree models found hardest, since convolutional filters can pick up local cues (collar shape, fabric folds, sleeve length) that a pixel threshold cannot represent. We would not expect the Shirt/Pullover/Coat/T-shirt confusion to vanish, since it is a known difficulty of this dataset even for deep models, but it should shrink well below Figure 6.

4 Conclusion

Moving from a single tree to a random forest cut test error by 38%, mostly by fixing the variance of individual trees, with the biggest gains on the hardest classes. What is left is concentrated in a handful of visually similar upper-body garments and comes from a representational limit of raw-pixel, axis-aligned splits rather than variance, so more trees will not fix it. A convolutional network targets that limit directly by learning spatial, translation-invariant features, which is why we propose it as the next step; published results on this dataset suggest it should meaningfully improve on our tree-based numbers, particularly for the classes identified above.

References

- H. Xiao, K. Rasul, R. Vollgraf. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv:1708.07747*, 2017.
- Zalando Research. Fashion-MNIST benchmark results. <https://github.com/zalando-research/fashion-mnist>, accessed September 2026.

A Per-class results and figures

Class	Decision tree	Random forest	Gradient boosting
T-shirt/top	0.765	0.843	0.848
Trouser	0.950	0.980	0.982
Pullover	0.684	0.784	0.820
Dress	0.814	0.892	0.905
Coat	0.668	0.793	0.817
Sandal	0.906	0.968	0.978
Shirt	0.518	0.649	0.691
Sneaker	0.891	0.940	0.958
Bag	0.918	0.967	0.979
Ankle boot	0.913	0.949	0.965
Macro F1	0.803	0.877	0.894

Table 3: Per-class F1 score on the test set for all three models. The upper-body garment classes (T-shirt/top, Pullover, Coat, Shirt) are the weakest for every model, though each successive model narrows the gap to the easier classes.

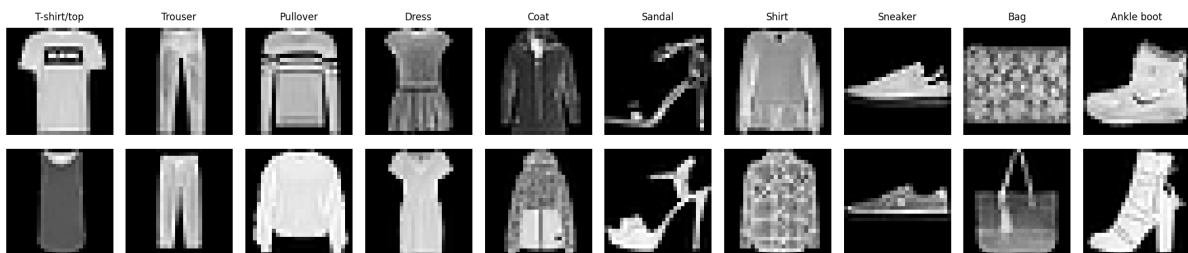


Figure 1: Two example images per class.

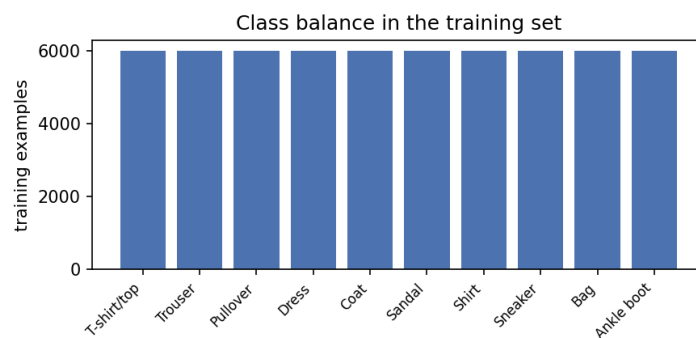


Figure 2: Training-set class balance (6,000 images per class; the test set has 1,000 per class).

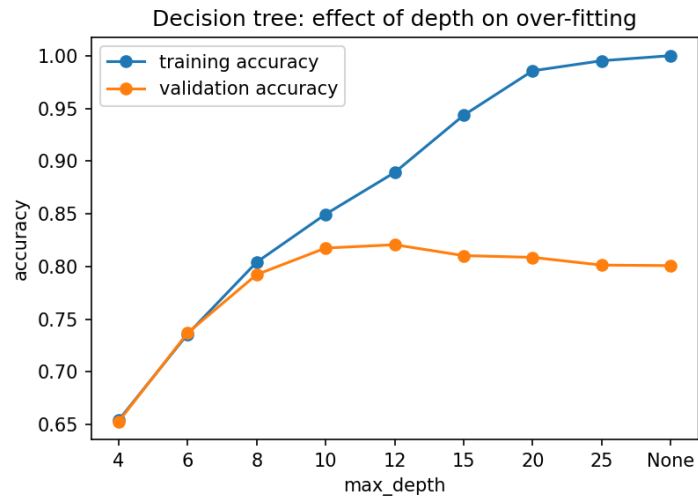


Figure 3: Decision tree: training vs. validation accuracy against `max_depth`.

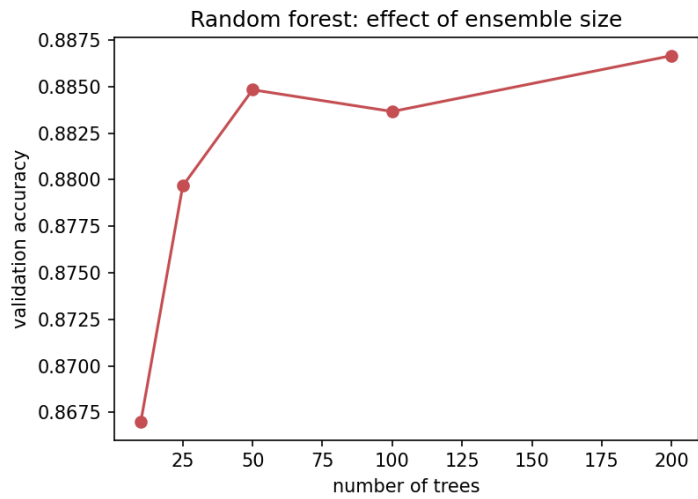


Figure 4: Random forest: validation accuracy against the number of trees.

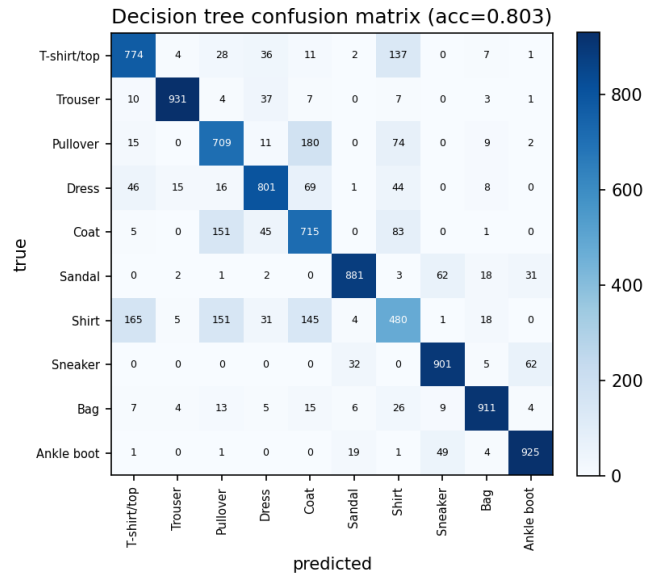


Figure 5: Decision tree confusion matrix (test set).

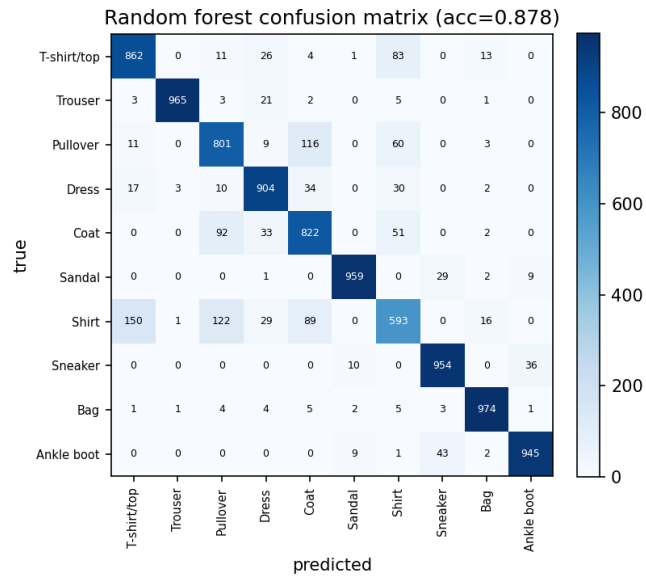


Figure 6: Random forest confusion matrix (test set).

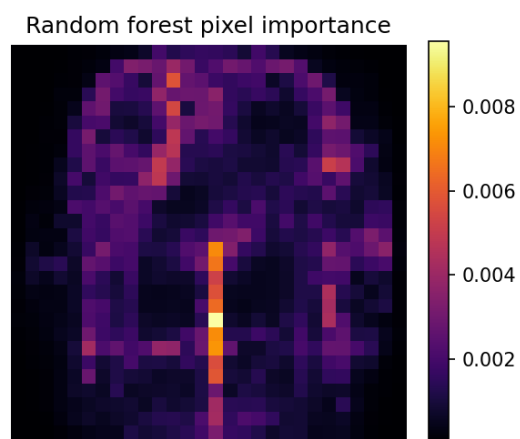


Figure 7: Random forest pixel importance, reshaped to 28×28 .

B How to run the code

The submitted `code.zip` contains `data_utils.py` (Fashion-MNIST loader) and `run_analysis.py` (full Part 1 pipeline), plus a `README.md` and `requirements.txt`. After installing the requirements (`pip install -r requirements.txt`) and placing the four Fashion-MNIST `idx.gz` files (available from the dataset repository) in a sibling `data/` folder, running `python run_analysis.py` from the `code/` folder reproduces every number and figure in this report end to end, writing results to `results/metrics.json` and `figures/`.